DOCKER MASTERY

Interview Prep

Level 2 — Topics 1, 2, 3

A focused interview prep document covering the foundations from Level 2 Topics 1, 2, and 3 — writing Dockerfiles, image layers and build caching, and the CMD vs ENTRYPOINT distinction.

40 questions ranked from basic to advanced, each with a structured answer in the language a senior engineer would use. Plus traps to watch for, bonus points to drop, and the answers worth memorizing word-for-word.

Read it. Then close it and say the answers out loud. The interview happens in your mouth, not on the page.

★ **Cross-cutting trick questions + how to prepare**

4 trap questions · practice exercises · what to memorize

How to use this document

This isn't a re-explanation of Topics 1, 2, and 3 — it's the **interview-shaped version** of what you already learned. The questions here are the ones that come up most often in real interviews for DevOps, SRE, and platform engineering roles, ordered roughly by difficulty.

The mindset before you answer

When an interviewer asks any Docker question, they're testing three things at once:

1. **Do you know multiple aspects of the answer** (not just one)?
2. **Can you reason about WHY each thing is the way it is** (not just memorize)?
3. **Do you know what to apply first** (priority and impact)?

Most candidates give a 10-second one-liner. A senior candidate structures a 60-second answer with definition, mechanism, tradeoff, and a concrete example. **That's what separates the interview signals.**

> **HOW TO USE THIS PDF**
>
> Read each question. Understand the structure of the answer (definition → mechanism → tradeoff → example). Close the PDF. Try to deliver the answer in your own words, out loud, against a 60-second timer. Where you stumble — that's the gap. Re-read and try again. **Memorized answers crumble at the first follow-up question. Internalized structure survives any follow-up.**

The structure inside each section

Each section follows the same pattern:

- **BASIC** questions — definitional, the "does this person know Docker at all" check. Green badge.
- **INTERMEDIATE** questions — mechanism and tradeoffs. Where most candidates start to slip. Amber badge.
- **ADVANCED** questions — nuance, edge cases, "why is it like this" depth. Where seniors prove themselves. Red badge.

After the question card, you'll see the structured answer in italics — that's the language to aim for. Below that, callout boxes flag traps to watch for and bonus points to drop.

Section 1 — Writing Dockerfiles (Topic 1)

What this section tests

Whether you actually understand the Dockerfile instructions — not just that they exist, but what each one does, when to use ADD vs COPY, why exec form matters, what build context is, what EXPOSE actually does. These are the foundational questions. Stumble here and you'll be marked junior regardless of how good you are at later topics.

BASIC

Q1 · BASIC

What is a Dockerfile?

The structured answer:

"A Dockerfile is a plain text file containing a sequence of instructions that tell Docker how to build an image. Each instruction creates a new layer in the resulting image. When you run docker build, Docker reads the Dockerfile top to bottom, executes each instruction, and produces an immutable image you can then run as containers. Think of it as a recipe — deterministic, reproducible, version-controllable in git alongside your code."

Q2 · BASIC

What does the FROM instruction do?

The structured answer:

"FROM specifies the base image you're building on top of. It must be the first instruction in any Dockerfile (except for ARG declarations or comments). The base image gives you a starting filesystem — for example, FROM python:3.11-slim starts with a minimal Debian image that has Python 3.11 installed. Everything you add in subsequent instructions stacks on top of that. You can also use FROM scratch for an empty starting point, which is what compiled-language images use to ship just a single binary."

Q3 · BASIC

What's the difference between COPY and ADD?

The structured answer:

"Both copy files from the build context into the image. The difference is ADD has two extra capabilities: it can fetch files from URLs, and it auto-extracts local tar archives. Those features sound useful but they cause problems — URL fetches break reproducibility because the remote file can change, and auto-extraction is surprising behavior nobody expects. The standard advice from Docker themselves is: always use COPY unless you specifically need ADD's auto-extraction. For URL fetches, use RUN curl or RUN wget instead, where the behavior is explicit."

BONUS POINT

If they ask "but I see ADD in many Dockerfiles, why?" — the honest answer is "legacy habits." Older tutorials used ADD reflexively. Modern best practice is COPY by default.

Q4 · BASIC

What does WORKDIR do?

The structured answer:

“WORKDIR sets the working directory for any subsequent RUN, CMD, ENTRYPOINT, COPY, or ADD instructions. If the directory doesn't exist, Docker creates it. Setting WORKDIR /app means any relative paths in later instructions are interpreted relative to /app, and when the container starts, that's where the shell or process begins. It replaces the bad pattern of writing RUN cd /app && ... everywhere — that doesn't even work properly because each RUN is a fresh shell.”

INTERMEDIATE

Q5 · INTERMEDIATE

What is the build context?

The structured answer:

"The build context is the directory you pass as the last argument to docker build — usually a dot, meaning the current directory. When you run docker build, the very first thing the Docker CLI does is bundle that entire directory into a tarball and ship it to the Docker daemon. The daemon then uses that tarball as the source for any COPY or ADD instructions. This matters because anything outside the build context is completely invisible to your build. If your Dockerfile says COPY ../src ./src, that fails — you can't COPY from outside the context. It also matters for build speed: a 500 MB build context takes seconds to upload before the build even starts, which is why .dockerignore is critical."

Q6 · INTERMEDIATE

What's the difference between ENV and ARG?

The structured answer:

"Both define variables, but their lifetimes are different. ARG is a build-time variable — it exists only while the image is being built and disappears when the build finishes. ENV is a runtime variable — it's baked into the image and present in every container that runs from it. You'd use ARG to parameterize the build, like ARG VERSION=1.0 followed by FROM python:${VERSION}-slim. You'd use ENV to set runtime configuration like ENV PYTHONUNBUFFERED=1. A common mistake is putting secrets in ARG thinking they disappear — they're visible in docker history and image layers if you reference them in a RUN. For secrets, use BuildKit's --mount=type=secret or runtime injection instead."

Q7 · INTERMEDIATE

What does EXPOSE actually do?

The structured answer:

*"Almost nothing — and that surprises most candidates. EXPOSE is purely documentation. It tells humans and tools that the image expects to listen on a particular port. It does **not** actually open or publish that port to the host. To make a container's port reachable, you need docker run -p 8080:5000 or the equivalent in Docker Compose or Kubernetes. If you want the strict answer: EXPOSE adds metadata to the image that docker run -P (capital P) can read to auto-publish all exposed ports to random host ports. But in practice, every real deployment uses explicit -p mappings, so EXPOSE is documentation."*

WATCH OUT

If they push back with "so why bother including EXPOSE at all?" — say it documents intent for whoever runs the image, and some orchestrators do read it for service discovery. It's not useless, just not what most people think it is.

Q8 · INTERMEDIATE

Why should you set a non-root USER in your Dockerfile?

The structured answer:

“Defense in depth. By default, processes inside a container run as root. If your application has a vulnerability — a remote code execution, a deserialization bug, anything that lets an attacker run their commands — they'll be running those commands as root inside the container. Combined with certain misconfigurations (privileged mode, sensitive volume mounts), that can escalate to root on the host. Adding USER appuser before CMD limits the blast radius. The cost is essentially zero — two lines, RUN useradd to create the user and USER to switch to it. There's almost no downside, and it eliminates an entire class of attack escalation. Senior engineers do this reflexively.”

ADVANCED

Q9 · ADVANCED

What's the difference between exec form and shell form for RUN, CMD, and ENTRYPOINT?

The structured answer:

“Exec form is a JSON array — RUN ["apt-get", "install", "-y", "curl"]. Shell form is a plain string — RUN apt-get install -y curl. The difference: shell form wraps the command in /bin/sh -c, which means a shell process becomes PID 1. Exec form runs the command directly, and your process becomes PID 1. This matters for two reasons. First, signal handling — when you docker stop, Docker sends SIGTERM to PID 1. Shell form means SIGTERM goes to /bin/sh, not your app, which doesn't propagate it. Your app gets killed by SIGKILL after the timeout, no graceful shutdown. Second, exec form doesn't do shell expansion, so $VARIABLES aren't expanded — you'd have to wrap in sh -c manually if you need that. The rule: always use exec form for CMD and ENTRYPOINT. For RUN it's usually fine to use shell form because RUN happens at build time and PID 1 doesn't matter.”

Q10 · ADVANCED

If your Dockerfile has multiple RUN, COPY, ENV instructions, in what order does Docker execute them?

The structured answer:

“Strictly top to bottom, with one important nuance: each instruction creates a new layer that's stacked on top of all previous layers. So when RUN apt-get install runs on line 5, it sees a filesystem that includes everything from FROM plus all the changes from lines 2 through 4. There's no parallel execution within a single Dockerfile — that's strictly sequential. Multi-stage builds are different — BuildKit can execute independent stages in parallel — but within one stage, it's sequential. The other thing worth knowing: COPY and ADD can invalidate the cache for all subsequent instructions if the source files change, even if the instruction text is identical. That's because Docker's cache key for COPY includes a checksum of the copied files.”

Q11 · ADVANCED

How do you pass build-time arguments without baking them into the image?

The structured answer:

“Three approaches depending on what you're trying to do. For non-secret build-time configuration, use ARG and pass with docker build --build-arg VERSION=1.0. ARGs disappear when the build finishes — they're not in the final image, though they do show up in docker history. For build-time secrets like a private package registry token, use BuildKit's secret mounts: DOCKER_BUILDKIT=1 docker build --secret id=mytoken,src=./token and inside the Dockerfile use RUN --mount=type=secret,id=mytoken cat /run/secrets/mytoken. The secret is mounted as a tmpfs during that one RUN and never lands in any layer. For SSH keys to clone

private repos, BuildKit has --mount=type=ssh which forwards your SSH agent. The rule: ARG for non-sensitive parameterization, BuildKit secret mounts for anything sensitive, never ENV or COPY for secrets at build time."

Q12 · ADVANCED

Why might a Dockerfile that builds successfully on your laptop fail in CI?

The structured answer:

"Several reasons, most of them caching or context related. First, your laptop has a populated layer cache so steps that would fail on a fresh build might be silently using cached layers — for example, if your Dockerfile uses RUN apt-get install without apt-get update in the same line, the cached package list might be valid locally but stale on a clean CI runner. Second, build context differences — your local .dockerignore might be missing files that exist on your machine but not in CI (or vice versa). Third, architecture mismatch — building on Apple Silicon and deploying to x86 without --platform linux/amd64. Fourth, base image floating tags — if you used FROM python:3.11 and the latest patch shifted, behavior changes. Fifth, secrets — your laptop has credentials cached that CI doesn't. The fix is build discipline: pin base images, .dockerignore everything, explicitly set --platform, and test fresh builds locally with docker build --no-cache."

Section 2 — Image Layers & Caching (Topic 2)

What this section tests

Whether you understand *why* Dockerfiles are written the way they are. Most candidates write Dockerfiles by copy-pasting patterns. A senior engineer can explain why each line is in the order it's in, what invalidates the cache, what's actually stored on disk, and how to debug a slow build. This is the single most discriminating section in a Docker interview.

BASIC

Q13 · BASIC

What is a Docker image layer?

The structured answer:

"A layer is a single read-only filesystem snapshot produced by one Dockerfile instruction. Every RUN, COPY, and ADD creates a new layer. The layer captures only the diff — what changed compared to the layer below it. When you run a container, Docker stacks all the layers into a single union filesystem and adds a thin writable layer on top for runtime changes. Layers are stored once and shared between images — if two images both use FROM python:3.11-slim, the base layers are downloaded and stored once, not twice."

Q14 · BASIC

What is the build cache?

The structured answer:

"Docker's build cache is the mechanism that skips work it has already done. When Docker is executing a Dockerfile and reaches an instruction, it checks: have I run this exact instruction, with these exact inputs, on top of this exact parent layer before? If yes, it reuses the existing layer instead of running the instruction again. That's why your second build is much faster than your first — most layers are cache hits. The cache is what makes Docker workflows tolerable; without it every build would be a full rebuild."

Q15 · BASIC

Why are images said to be immutable?

The structured answer:

"Once a layer is built, it cannot be modified. Period. If you want to change a file in an image, you don't edit the layer — you build a new image with a new layer that overwrites or removes that file, but the original layer still exists unchanged. This is fundamental to how Docker works. It's why deleting a large file in a later RUN doesn't actually save space — the file's bytes remain in the earlier layer, and the deletion is just a whiteout marker in the new layer that hides the file at runtime. To actually save space, the create-and-delete must happen in the same RUN."

INTERMEDIATE

Q16 · INTERMEDIATE

When is the build cache invalidated?

The structured answer:

"Three triggers, in order of frequency. First, when the instruction text changes — even a whitespace change in a RUN line invalidates that line's cache and everything below it. Second, when the source files for COPY or ADD change — Docker checksums the contents, not just timestamps, so changing a file's contents invalidates the cache. Third, when a parent layer is invalidated — once any line cache-misses, every subsequent line in the Dockerfile is also a cache-miss regardless of whether they would otherwise hit. There's also an implicit fourth trigger: docker build --no-cache forces a full rebuild. The cascading nature of cache invalidation is why ordering matters so much."

Q17 · INTERMEDIATE

What is the golden rule of Dockerfile ordering?

THE BIG ONE

This is one of the most asked Docker interview questions. Memorize the structure of this answer.

The structured answer:

"Order instructions from least frequently changing to most frequently changing. The reasoning is direct: when something changes, every layer below the change invalidates and rebuilds. Source code changes way more often than dependency lists, and dependency lists change way more often than the base image. So the canonical pattern is: FROM at the top, then system-level RUN apt-get installs, then COPY of dependency manifests like requirements.txt or package.json, then RUN to install dependencies, then finally COPY of the application source code. With that ordering, when you change a Python file, only the last COPY and the CMD layers invalidate — the dependency install stays cached, which is the slowest step. Get this wrong and your dev loop has 60-second waits instead of 2-second waits."

Q18 · INTERMEDIATE

Walk me through how this Dockerfile uses the cache. What changes when I edit a Python file?

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY src/ src/
CMD ["python", "src/app.py"]
```

The structured answer:

"Walking through it: Line 1 FROM hits the cache because the base image hasn't changed. Line

2 WORKDIR is cached. Line 3 COPY requirements.txt — Docker checksums requirements.txt; if it's identical to the previous build, this hits cache. Line 4 RUN pip install — cached because line 3 was a cache hit. Line 5 COPY src/ src/ — Docker checksums everything in src/; because we edited a Python file, the checksum changes and this is a cache miss. Line 6 CMD is also a miss because line 5 missed. So editing a Python file rebuilds only the last two layers. The expensive pip install stays cached. That's the golden rule paying off — separating COPY of dependencies from COPY of source code lets each invalidate independently."

Q19 · INTERMEDIATE

Why should you chain RUN commands with &&?

The structured answer:

"Two reasons. First, fewer layers — each RUN creates a layer, and each layer adds metadata overhead. Combining related commands into one RUN keeps the layer count down. Second, and more importantly, cleanup actually saves space when it's in the same RUN. The classic example: RUN apt-get update on one line followed by RUN apt-get install -y curl on the next leaves the apt cache permanently in the image, because the update layer is committed before the install layer can clean up. Combining into RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/ puts everything in one layer that gets committed only after the cleanup runs — the cache files never make it into the layer."*

Q20 · INTERMEDIATE

Where does Docker store image layers on disk?

The structured answer:

"On Linux it's typically /var/lib/docker/, with the storage driver determining the exact subdirectory. Modern Docker uses overlay2 by default, so layers live under /var/lib/docker/overlay2/. Each layer is a directory containing the diff filesystem and metadata. On Mac and Windows, Docker Desktop runs a Linux VM and the storage is inside that VM. The relevant commands are docker system df to see space used by images, containers, and volumes, and docker system prune to clean up dangling layers and stopped containers. Layers from a removed image aren't immediately freed if any other image references them — that's why deleting a 1 GB image sometimes only frees 50 MB."

Q21 · INTERMEDIATE

What does --no-cache do, and when would you use it?

The structured answer:

"docker build --no-cache forces every instruction to re-run from scratch, ignoring the cache entirely. The use cases are specific. First, when you suspect the cache is hiding a real problem — for example, a broken apt-get update that's still serving cached data. Second, before pushing to production, to confirm the build still works on a fresh environment matching what CI will see. Third, when debugging a flaky build to isolate cache-related issues. You wouldn't use --no-cache routinely — it makes builds 10x slower and wastes electricity. It's a debugging tool,

not a development habit.”

ADVANCED

Q22 · ADVANCED

What is BuildKit and why does it matter?

The structured answer:

"BuildKit is the modern Docker build engine. It replaced the legacy builder around 2018 and became default in Docker 23. It brings several improvements that matter in interviews. First, parallel stage execution — independent stages in a multi-stage build run concurrently instead of sequentially, often cutting build time significantly. Second, better caching — BuildKit can cache to remote registries, share cache across machines, and use mount-based caches for package managers. Third, secret mounts — --mount=type=secret and --mount=type=ssh let you use credentials at build time without baking them into layers. Fourth, more efficient context transfer — only changed files are sent to the daemon. To enable on older Docker, set DOCKER_BUILDKIT=1 before docker build. On modern Docker it's already on by default."

Q23 · ADVANCED

How does Docker decide if a COPY instruction is a cache hit?

The structured answer:

"Docker computes a hash of the source files being copied — the actual file contents, not just timestamps or names. That hash, combined with the instruction text and the parent layer's hash, becomes the cache key. If the cache key matches a previously built layer, it's a hit. The implication is subtle: you can change a file's modification time, rename the source folder back and forth, or even copy files in and out — as long as the actual byte contents and structure of what gets copied are identical, the cache hits. Conversely, even a one-byte change to any copied file is a cache miss. This is also why .dockerignore matters for caching — files that don't need to be in the build context shouldn't be there, because their changes can invalidate COPY layers."

Q24 · ADVANCED

If I run RUN apt-get update without apt-get install in the same line, what's the problem?

The structured answer:

"The cache becomes a liar. apt-get update creates a layer with the package index from whatever date the build happened. That layer gets cached. Months later, you add a new RUN apt-get install -y newpackage line below it. Docker sees the apt-get update line is unchanged, hits the cache, and reuses the months-old package index. Then apt-get install runs against that stale index — and either fails to find newer package versions, or installs an old vulnerable version. The fix is always to combine update and install in the same RUN: RUN apt-get update && apt-get install -y package && rm -rf /var/lib/apt/lists/. That way any change to the install list invalidates the update too, and the index is always fresh when packages get installed."*

Q25 · ADVANCED

How would you debug a Dockerfile that's building slowly?

The structured answer:

“Systematic approach. First, run with docker build --progress=plain to see timing per layer instead of the summarized output — you'll see exactly which instructions are slow. Second, look for cache misses on layers that should be cached. If pip install is rebuilding every time, your COPY of requirements.txt is happening after a COPY . . or your requirements.txt is changing. Third, check the build context size — if 'transferring context' shows hundreds of MB, your .dockerignore is missing or incomplete. Fourth, check for a chained RUN with an expensive download — apt-get installs of large packages take time. Fifth, use docker history myimage to see the size and command of each layer; suspiciously large layers often point to unnecessary cache or temp files. The tool dive lets you explore layer contents interactively if you need to go deeper.”

Q26 · ADVANCED

What's the difference between docker history and docker inspect?

The structured answer:

“Different views of the same image. docker history shows the layers — what command created each one, how big each is, when each was created. It's how you'd answer questions like "why is this image so big" or "was a secret leaked into a layer." docker inspect shows the metadata — environment variables, exposed ports, the CMD, ENTRYPOINT, USER, WORKDIR, labels, and the digest. It's how you'd answer "what command does this run by default" or "what user does it run as." In an interview, history for layers and size, inspect for configuration. Both work on running containers too with different flags.”

Section 3 — CMD vs ENTRYPOINT (Topic 3)

What this section tests

Whether you understand the most-confused pair of Dockerfile instructions. Most candidates have a vague sense that "CMD is the default command and ENTRYPOINT is something else." Senior engineers can explain the override rules, when to combine them, why exec form matters for signal handling, and what a tini-style init does. This is also where small mistakes cause real production incidents — apps that don't shut down gracefully, containers that ignore docker stop, weird argument-passing behavior.

BASIC

Q27 · BASIC

What's the difference between CMD and ENTRYPOINT?

The structured answer:

“Both define what runs when a container starts, but with different override semantics. CMD provides default arguments or a default command that can be completely replaced by passing arguments to docker run. ENTRYPOINT defines the main executable that always runs — arguments to docker run are appended to it instead of replacing it. So CMD makes the container act like a default invocation that's easy to override; ENTRYPOINT makes the container act like a single command-line tool. The most common pattern is to combine them: ENTRYPOINT defines the binary, CMD defines the default arguments. Then docker run myimage uses the defaults, and docker run myimage --custom-flag passes through to the binary.”

Q28 · BASIC

If a Dockerfile has both CMD and ENTRYPOINT, which one runs?

The structured answer:

“Both, combined. ENTRYPOINT is the executable, CMD is the default arguments passed to it. So if the Dockerfile has ENTRYPOINT ["nginx"] and CMD ["-g", "daemon off;"], the container runs nginx -g "daemon off;". When someone runs docker run myimage --different-flag, the CMD is replaced and the container runs nginx --different-flag — but ENTRYPOINT stays. That's the design intent: ENTRYPOINT locks in the command, CMD is the easily-replaceable default.”

Q29 · BASIC

What does docker run myimage echo hello do?

The structured answer:

“It depends on what's in the Dockerfile. If the Dockerfile has only CMD, the CMD is replaced and the container runs echo hello. If the Dockerfile has ENTRYPOINT ["python"] and CMD

["app.py"], then the CMD is replaced and the container runs python echo hello — which is a Python error because echo isn't a Python file. If the Dockerfile has only ENTRYPOINT and you want to override that, you need docker run --entrypoint=echo myimage hello. The takeaway: arguments to docker run replace CMD by default, never ENTRYPOINT — to override ENTRYPOINT you need the explicit --entrypoint flag."

INTERMEDIATE

Q30 · INTERMEDIATE

What's the difference between exec form and shell form for CMD?

The structured answer:

“Exec form is the JSON array — CMD ["python", "app.py"]. Shell form is the plain string — CMD python app.py. The difference is what becomes PID 1 in the container. Exec form runs your command directly; your python process is PID 1. Shell form is equivalent to CMD ["sh", "-c", "python app.py"]; the shell becomes PID 1 and your Python process is a child. This matters for signals: when Docker sends SIGTERM on docker stop, it goes to PID 1. Exec form means your app receives SIGTERM and can shut down gracefully. Shell form means /bin/sh receives SIGTERM, which doesn't forward signals to its children by default. Your app gets SIGKILL'd after the 10-second grace period instead, skipping any cleanup. The rule: always use exec form for CMD and ENTRYPOINT in production.”

Q31 · INTERMEDIATE

Why does signal handling matter, and how does it relate to PID 1?

The structured answer:

“Containers communicate the "please shut down" intent through signals. When orchestrators terminate a container — Kubernetes rolling deploy, autoscaler scale down, docker stop — they send SIGTERM first, wait a grace period (typically 10 to 30 seconds), then send SIGKILL if the container hasn't exited. SIGTERM is your app's opportunity to gracefully shut down: finish in-flight requests, close database connections, flush buffers, release locks. SIGKILL is unconditional — the kernel kills the process immediately, no cleanup runs. Now, signals only go to PID 1, the process that started the container. If PID 1 is your app, you receive SIGTERM and can handle it. If PID 1 is a shell wrapping your app, the shell receives SIGTERM and ignores it, and your app gets killed without warning. That's why exec form matters.”

Q32 · INTERMEDIATE

When would you use ENTRYPOINT and CMD together vs just CMD?

The structured answer:

“Use just CMD when the image is a general-purpose runtime that someone might want to override completely. A development image where you might docker run it bash to explore — CMD is the right choice because users naturally want to swap it. Use ENTRYPOINT plus CMD when the image represents a specific command-line tool. Take an image that wraps the AWS CLI: ENTRYPOINT ["aws"] locks in the binary, CMD ["--help"] is the default action. Now docker run myimage s3 ls works naturally — the s3 ls is appended to aws. The user can't accidentally docker run myimage rm -rf / because ENTRYPOINT prevents that override. The rule of thumb: if your image is a tool, use both. If your image is a runtime environment, use just CMD.”

Q33 · INTERMEDIATE

What's a common mistake people make with ENTRYPOINT?

The structured answer:

“The biggest one: using shell form. ENTRYPOINT python app.py (shell form) wraps in /bin/sh -c, which means signals don't propagate, your app doesn't shut down cleanly, and additionally, CMD arguments get ignored entirely — the shell form ENTRYPOINT swallows them. So docker run myimage extra-arg behaves nothing like the user expects. The fix is always exec form: ENTRYPOINT ["python", "app.py"]. The second common mistake is hardcoding all the arguments into ENTRYPOINT and leaving CMD empty — which works, but removes the user's ability to override defaults via docker run. Best practice is to put the binary in ENTRYPOINT and the default arguments in CMD.”

ADVANCED

Q34 · ADVANCED

What is tini, dumb-init, or an init system in a container, and when do you need one?

The structured answer:

“Tools like tini and dumb-init are minimal init systems designed for containers. They solve two problems that happen when your application is PID 1 directly. First, PID 1 has special semantics in Linux — it doesn't get the default signal handlers, so if your app doesn't explicitly handle SIGTERM and SIGINT, those signals are ignored entirely. Second, PID 1 is responsible for reaping zombie processes — child processes that have exited but whose status hasn't been collected. If your app spawns subprocesses (shell scripts, language runtimes that fork) and doesn't reap them, zombies accumulate. The fix is to put a tiny init like tini at PID 1 and make it spawn your app. Tini handles signal forwarding and zombie reaping properly, then your app is PID 2 and behaves normally. Use it when your app spawns subprocesses or doesn't have explicit signal handlers. Modern Docker has docker run --init that adds a built-in tini automatically.”

Q35 · ADVANCED

If I write CMD ["echo", "$HOME"], what gets printed?

The structured answer:

“Literally the string $HOME, not the value of the home variable. That's because exec form doesn't go through a shell — there's no shell to do variable expansion. To get the variable's value, you need shell form: CMD echo $HOME, or you need to invoke a shell explicitly: CMD ["sh", "-c", "echo $HOME"]. This is a common gotcha. If your CMD or ENTRYPOINT needs environment variable expansion, you have to choose: shell form (loses signal handling) or exec form with sh -c wrapping (regains shell features but still has the PID-1 signal problem). The cleaner solution is usually to do the expansion in your application code itself, not in the Dockerfile.”

Q36 · ADVANCED

How do you make a container that behaves like a CLI tool?

The structured answer:

“Use ENTRYPOINT with the binary in exec form, optionally with CMD providing default arguments. Take wrapping curl: ENTRYPOINT ["curl"], CMD ["--help"]. Now docker run mycurl behaves like curl --help. docker run mycurl https://example.com behaves like curl https://example.com. If you also need shell features inside the entrypoint, use an entrypoint script — a small shell script that you COPY in, make executable, and reference: ENTRYPOINT ["/usr/local/bin/entrypoint.sh"]. The script can do setup work like waiting for the database, applying migrations, then exec your real binary at the end. The exec at the end is critical — it replaces the script process with your app, so your app becomes PID 1 with proper signal

handling.”

Section 4 — Cross-cutting trick questions

Why these matter

These questions don't fit one topic — they test whether you understand connections across topics. They're often the deciding questions in senior interviews because they reveal whether you've internalized the model or just memorized facts.

Q37 · ADVANCED

What's the difference between a Docker image and a container?

The structured answer:

"An image is a static, read-only template — a stack of layers plus metadata describing how to start a container from it. An image doesn't run anything; it just sits on disk. A container is a running instance of that image, with its own isolated process namespace, network namespace, filesystem, and resources. Compare to OOP: image is the class, container is the instance. You can have many containers running from the same image, each with their own runtime state, completely independent. When the container stops, its writable layer is preserved (until docker rm), but the image itself is unchanged — that's the immutability property again. The CMD inside the image only executes when a container is created and started, not when the image is built."

Q38 · ADVANCED

If I add 'RUN echo secret > /tmp/key' and then 'RUN rm /tmp/key', is the secret in my final image?

The structured answer:

"Yes. This is the layer immutability question disguised. The first RUN creates a layer that contains /tmp/key with the secret content. That layer is immutable. The second RUN creates a new layer with a whiteout marker hiding /tmp/key from the filesystem view, but the underlying bytes are still in the previous layer. Anyone with access to the image can extract individual layers and read the secret. Even worse: anyone who runs docker history sees the literal command 'echo secret > /tmp/key' if you used shell form. The only correct way is to put both operations in a single RUN — RUN echo secret > /tmp/key && use_secret && rm /tmp/key — so the file never gets committed to a layer. Better: don't use the file approach at all, use BuildKit secret mounts."

WHAT THIS TESTS

Whether you actually understand layers. Most beginners say "no, it's removed." A senior engineer says "no, but it's still in the layer, and it's worse than that — it's in docker history too."

Q39 · ADVANCED

Why does this Dockerfile rebuild from scratch every time I change app.py?

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

The structured answer:

“The COPY . . happens before pip install. So when you change app.py, the COPY layer's checksum changes, that layer is a cache miss, and every layer below it — including the expensive pip install — also misses cache. The fix is the golden ordering rule: copy the dependency manifest first, install dependencies, then copy the source code. So: COPY requirements.txt . on its own line, RUN pip install on the next, then COPY . . at the end. After the fix, changing app.py only invalidates the final COPY and CMD — pip install stays cached. This single reorder typically takes a 60-second iteration loop down to 2 seconds.”

Q40 · ADVANCED

If I run the same Dockerfile twice in a row with no changes, will the resulting images be byte-identical?

The structured answer:

“Almost always no, and that surprises most candidates. Even with full cache hits, several things break exact byte equality. Timestamps embedded in package files might differ. RUN commands with non-deterministic output — anything that touches dates, random numbers, or network responses — produces different bytes. Floating tags like FROM python:3.11 might resolve to a different patch version. Build context tar packing isn't strictly deterministic across runs unless you control file timestamps. Even with cache hits, the image config manifest includes a created timestamp. The image SHA may match if every layer is a cache hit, but a from-scratch rebuild typically produces a different SHA. For real byte-identical reproducibility you need pinned base images by digest, locked dependencies, SOURCE_DATE_EPOCH-aware tooling, and BuildKit's reproducible mode. Most teams accept "same source, same dependencies, same behavior" as enough — true byte-equality is supply-chain-paranoia territory.”

How to actually prepare

Reading these answers isn't preparation. **Saying them out loud is preparation.** The interview is a verbal performance, and verbal performance has to be practiced verbally.

Exercise 1 — The 60-second drill

Pick five questions at random from this document. Set a timer for 60 seconds per question. Without notes, deliver an answer out loud. Record yourself if you can. Listen back. Where do you stumble, where do you wander off track, where do you repeat yourself? Those are your gaps. Re-read the structured answer, then try again. Repeat until you can deliver a clean 60-second answer for any random question.

Exercise 2 — The follow-up drill

After answering a question, ask yourself: what would the interviewer follow up with? Then answer that. Then the follow-up to the follow-up. Senior interviewers drill three or four levels deep. If you stop at the first answer, you're a junior. If you have three levels of depth ready, you're a senior.

Worked example. They ask: *What's the difference between CMD and ENTRYPOINT?* You give the structured answer. They follow up: *So when would you use both together?* You answer Q32. They follow up: *What's a common mistake with ENTRYPOINT?* You answer Q33. They follow up: *OK, but if exec form is so important, why does shell form exist?* Now you're at level four — and the senior answer is: shell form is convenient when you need shell features like variable expansion or piping during build-time RUNs, where PID 1 doesn't matter. The mistake is using it for CMD or ENTRYPOINT.

Exercise 3 — The inverse drill

Cover the question, read only the answer, and try to reconstruct the question. This forces you to notice the structure of how senior engineers explain things — the definition-mechanism-tradeoff-example pattern. Once that structure is internalized, you can apply it to any question, including ones you didn't prepare for.

The five answers worth memorizing word-for-word

Most answers should be in your own words. But these five are asked so often, and have such a specific structure, that rehearsing them word-for-word gives you confidence that transfers to the rest:

Q9 — What's the difference between exec form and shell form?
Tests whether you understand PID 1 and signal handling. Asked in nearly every Docker interview.

Q17 — What is the golden rule of Dockerfile ordering?
The single most discriminating Topic-2 question. The structured answer signals senior immediately.

Q19 — Why should you chain RUN commands with &?
Tests layer immutability understanding. The cleanup-in-same-layer reasoning is the senior signal.

Q22 — What is BuildKit and why does it matter?
Modern Docker question. Mentioning parallel stages, secret mounts, and remote cache hits all the senior talking points.

Q31 — Why does signal handling matter, and how does it relate to PID 1?
Production-readiness question. The graceful-shutdown story is the senior framing.

The traps to remember

The questions where most candidates default to a wrong answer and senior candidates pause. If you know these traps, you can give the right answer reflexively:

✗ EXPOSE doesn't actually publish ports — it's just metadata.

Only docker run -p actually maps ports. EXPOSE is documentation.

✗ ADD vs COPY — use COPY by default.

ADD's URL fetching and tar extraction features cause more problems than they solve. COPY is explicit.

✗ Deleting a file in a later RUN doesn't free space.

Layers are immutable. The bytes remain in the earlier layer. Combine create-and-delete in the same RUN.

✗ RUN apt-get update without install in the same line is a bug.

Stale package indexes get cached, leading to mysterious package version issues months later.

✗ Shell form for CMD breaks signal handling.

/bin/sh becomes PID 1 and doesn't forward signals. Always use exec form for CMD and ENTRYPOINT.

✗ ENV doesn't expand in exec-form CMD.

CMD ["echo", "$HOME"] prints literally $HOME, not the home directory.

✗ .dockerignore doesn't retroactively remove files from pushed images.

Once an image with a secret is pushed, it's compromised forever. Rotate the secret, don't just rebuild.

Final advice for the interview itself

WHEN YOU DON'T KNOW THE ANSWER

Don't bullshit. Say "I haven't worked with that specifically, but based on what I know about [related concept], I'd guess..." That's a senior move. Confidently making things up is a junior move that experienced interviewers spot instantly. **Honesty + reasoning under uncertainty is what they're actually testing.**

STRUCTURE EVERY ANSWER

Definition first. Then mechanism (how it works). Then tradeoffs (when to use, when not to use). Then a concrete example or numbers if you have them. **This four-beat structure works for any Docker question, even ones you haven't seen before.**

WHAT NOT TO SAY

Don't say "I think" or "I'm not sure but" before factual answers — say it confidently or don't say it. Don't pad answers with filler like "great question." Don't say "Docker is a containerization platform" — that's a junior tell. Don't apologize for not knowing something — just say you don't know it and reason from what you do know.

Your goal in 60 seconds

When the interviewer asks any question, your goal in the first 60 seconds is to make them think: **this person has actually built and shipped containers in production**. The answers in this document are written in the language of someone who has — confident on the basics, nuanced on the tradeoffs, willing to push back when the question has a wrong assumption baked in. That's the signal you're trying to send.

LEVEL 2 INTERVIEW PREP COMPLETE

■ You now have structured answers for every question that comes up most often on Topics 1, 2, and 3. Combined with the Topics 4-6 interview prep, you're prepared for the full Dockerfile-and-image surface area. **Practice out loud before the interview.** The interview is a performance, and performances are won in rehearsal.